

The Block-MiniJava Workbench: UI Design and Layout

Shell Architecture, Perspectives, Panel Sizing, and Responsive Behavior

Research note — Block-MiniJava

2026-07-20

Abstract

Block-MiniJava presents itself as a small IDE, not a single Blockly canvas with a toolbar bolted on. Its shell — header, activity bar, primary sidebar (toolbox), workspace column, inspector column, bottom dock, and status bar — is a genuine *workbench*, in the sense the README uses the word: named *regions* (`data-ide-region` attributes), *perspectives* that reconfigure several regions atomically, a keyboard-first *command palette*, and layout state that persists across reloads. This note documents that shell’s structure and every mechanism behind it — panel visibility and sizing, the perspective system, responsive breakpoints, and persisted state — tracing each to its file and line. The three content panels the shell hosts (inspector-column tabs, bottom-panel tabs, and the toolbox) are documented separately in `typing-code-outline-tabs.tex`, `bottom-panel-sync.tex`, and `toolbox-drag-and-drop.tex`.

Contents

1	Layout regions	1
2	Activity bar and primary sidebar	2
3	Perspectives	3
3.1	Why perspectives need a re-entrancy guard	4
4	Panel visibility and sizing	4
5	The layout-resize coordinator	5
6	Command palette	5
7	Status bar	6
8	Responsive breakpoints	6
9	Theming	7
10	Persisted state	7
	Note on scope	8

1 Layout regions

`src/index.html` tags every major structural element with a `data-ide-region` attribute — not read by any script (there is no region-dispatch logic keyed off it), but a deliberate naming convention that keeps the DOM self-documenting and gives CSS a stable, semantic hook independent of element ids.

Region	Element	Purpose
shell	#app (l.11)	The whole application root.
application-header	header.app-header (l.12)	Brand, command palette trigger, perspective picker, main menu (New/Open/Save/Export/Visualize/Autosave/Examples/About/theme toggle).
workbench	.ide-workbench (l.85)	Everything below the header: activity bar + the editor area.
activity-bar	#activity-bar (l.86)	The narrow icon strip selecting which sidebar view is showing (§2).
editor-area	main.ide-layout (l.105)	Primary sidebar + workspace column + inspector column, side by side.
primary-sidebar	#toolbox-column (l.106)	The toolbox (documented in <code>toolbox-drag-and-drop.tex</code>) and the other activity-bar-selected views (Run/Settings).
workspace	.workspace-column (l.168)	The Blockly canvas and its title-bar toolbar (undo/redo, zoom, run, toggle bottom tools, toggle inspector).
inspector	#code-column (l.210)	Editable Code / Typing / Outline (<code>typing-code-outline-tabs.tex</code>).
bottom-tools	#viz-dock (l.243)	The seven-tab dock (<code>bottom-panel-sync.tex</code>).
status-bar	footer.ide-status-bar (l.381)	Perspective button, block count, problems count, autosave status (§7).

2 Activity bar and primary sidebar

The activity bar is a four-icon vertical strip (`index.html:86--103`) that selects which *view* the primary sidebar shows — the same “activity bar + views” idea as a typical IDE’s left rail, implemented from scratch (no framework).

Symbol	Role
ActivityKind ACTIVITY_META	<code>app.ts:71</code> — ‘blocks’ ‘search’ ‘run’ ‘settings’. <code>app.ts:76--81</code> — per-activity {title, icon} shown in the sidebar header when that activity is active.
<code>setActiveActivity(activity, ensureVisible, focusSearch)</code>	<code>app.ts:439--456</code> — persists the choice (<code>ACTIVE_ACTIVITY_KEY</code>), updates the sidebar title/icon, toggles <code>is-active</code> on every <code>.sidebar-view[data-activity-view]</code> whose <code>data-activity-view</code> list includes the new activity (one view, <code>sidebar-blocks-view</code> , is shared between ‘blocks’ and ‘search’ — <code>index.html:114</code> , since Search Blocks is really the same toolbox view with the search box pre-focused, not a separate panel), reveals the sidebar if it was hidden, and auto-focuses <code>#toolbox-search</code> for the ‘search’ activity.

Symbol	Role
<code>updateActivityButtons()</code>	<code>app.ts:429--437</code> — toggles <code>is-active/aria-pressed</code> on the four <code>.activity-item</code> buttons; <code>aria-pressed</code> also factors in whether the sidebar is actually visible on narrow screens (§8), not just which activity is logically selected.

The three non-’blocks’ sidebar views are: **Run and Analysis** (`index.html:122--135` — a **Run Program** button plus five shortcuts straight into specific bottom-panel tabs), and **Settings and Layout** (`index.html:137--161` — the perspective picker’s button form, layout toggles, and the autosave interval slider). The toolbox view itself (`index.html:114--120`) is documented in `toolbox-drag-and-drop.tex`.

3 Perspectives

A *perspective* is a named, one-click preset over several independent layout toggles at once — sidebar visibility, inspector visibility/tab, bottom-dock visibility/tab, and code-maximize state — so a user does not have to manually reconstruct “the debugging layout” every time.

Perspective	What <code>applyPerspective</code> (<code>app.ts:485--521</code>) sets
<code>edit</code>	Activity ’blocks’; sidebar shown; inspector shown, not maximized, Editable Code tab; bottom dock closed . The default, block-authoring-focused layout.
<code>debug</code>	Activity ’run’; sidebar shown; inspector shown, Outline tab; bottom dock open on CESK .
<code>types</code>	Activity ’blocks’; sidebar shown; inspector shown, Outline tab; bottom dock open on Problems , and <code>refreshTypeDiagnostics</code> is called immediately so the Problems list is not stale on arrival.
<code>presentation</code>	Code column and sidebar both hidden , code not maximized, viz dock closed — a maximized, distraction-free block workspace for demoing or screenshotting.
<code>custom</code>	Not a preset — see below.

Symbol	Role
<code>setPerspectiveIdentity(perspective, persist)</code>	<code>app.ts:458--474</code> — the <i>identity</i> update only: sets <code>document.body.dataset.perspective</code> , toggles the <code>presentation-mode</code> body class, syncs the <code>#perspective-select</code> dropdown and the perspective-option buttons’ <code>is-active</code> state, and updates the status bar’s perspective label (§7). Does <i>not</i> touch any panel visibility — that is <code>applyPerspective</code> ’s job.
<code>applyPerspective(perspective)</code>	<code>app.ts:485--521</code> — sets <code>applyingPerspective = true</code> (a re-entrancy guard, §3.1) for its whole body, calls <code>setPerspectiveIdentity</code> , then the table above’s panel toggles.

Symbol	Role
<code>markPerspectiveCustom()</code>	<code>app.ts:476--478</code> — called by every manual panel-visibility toggle (sidebar hide/show, code hide/show/maximize, viz-dock open/close); if <code>applyingPerspective</code> is <code>false</code> (i.e. the change came from the user directly, not from <code>applyPerspective</code> itself reconfiguring things) and the current perspective is not already <code>'custom'</code> , switches identity to <code>'custom'</code> .

3.1 Why perspectives need a re-entrancy guard

Every one of the four preset perspectives is implemented by calling the *same* panel-visibility setters (`setToolboxHidden`, `setCodeHidden`, `openBottomTool`, ...) that a user's manual click also calls. Those setters cannot tell, by themselves, whether they were invoked because the user just clicked **Hide inspector** or because `applyPerspective('debug')` is reconfiguring five things at once as an atomic preset. `applyingPerspective` is exactly that missing context: `applyPerspective` sets it before making any change and clears it in a `finally` block (`app.ts:517--521`) so an exception mid-preset cannot leave the flag stuck; every setter's trailing call to `markPerspectiveCustom` then correctly no-ops during a preset application and correctly demotes the identity to `'custom'` on any *later*, genuinely manual adjustment — matching the design note's own description: "Adjusting panels by hand switches to `Custom` rather than misreporting a preset."

4 Panel visibility and sizing

Function	Role
<code>setToolboxHidden(next)</code>	<code>app.ts:644--664</code> — toggles the <code>toolbox-hidden</code> body class; on narrow screens toggles <code>mobile-sidebar-open</code> instead (§8); persists to <code>SIDEBAR.VISIBLE_KEY</code> ; updates the show/hide button labels and <code>updateActivityButtons()</code> ; calls <code>requestLayoutResize()</code> (§5).
<code>setCodeHidden(next)</code>	<code>app.ts:620--641</code> — the inspector-column analogue; also un-maximizes the code column if it was maximized (a hidden panel cannot be maximized).
<code>setCodeMaximized(next)</code>	<code>app.ts:607--618</code> — toggles the <code>code-maximized</code> body class (CSS grows the inspector column to fill the editor area); reveals the inspector first if it was hidden (l.609); swaps the maximize button's glyph between a hollow-square and an offset-square icon.
<code>setVizOpen(open)</code>	documented in <code>bottom-panel-sync.tex</code> §1 — visualizationPanel.ts:157--173.
<code>setSidebarWidth(width)</code>	<code>app.ts:944--950</code> — clamps to <code>[220px, min(420, 45% viewport)]</code> , writes the CSS custom property <code>--ide-primary-sidebar-width</code> , persists to <code>SIDEBAR.WIDTH_KEY</code> .
<code>initSidebarResizer()</code>	<code>app.ts:952--(follows)</code> — pointer-drag on <code>#sidebar-resizer</code> , mirroring the code-column and viz-dock resizers' pointer-capture pattern.

All three resizable regions — primary sidebar, inspector column, bottom dock — follow the identical resizer pattern: a thin `role="separator"` element with `pointerdown` starting a `pointermove/pointerup` drag (removed on release), plus `ArrowUp/ArrowDown` or `ArrowLeft/ArrowRight` keyboard resizing for accessibility, each clamped to a sane range and persisted independently.

5 The layout-resize coordinator

Blockly's own SVG canvas, the code editor's line-number/highlight overlay, and every visualization tab's own Blockly instance all need to be told explicitly when their container changes size — none of this is automatic in the browser. `app.ts` centralizes that into one coordinator rather than letting each panel poll or guess.

Function	Role
<code>syncLayoutSize()</code>	<code>app.ts:89--95</code> — the actual work: <code>Blockly.svgResize(workspace)</code> , <code>codeEditor?.resize()</code> , <code>resizeVisualizationPanel()</code> .
<code>requestLayoutResize(settle)</code>	<code>app.ts:101--114</code> — the coordinator every panel-visibility setter calls. One <code>requestAnimationFrame</code> handles the immediate case (covers pointer-driven resizing, which fires continuously); when <code>settle</code> is true (the default), <i>two more</i> calls are scheduled at 60ms and 180ms via <code>setTimeout</code> to catch CSS grid <i>transition</i> end-states a single animation frame would miss (a CSS-animated width change keeps changing after the triggering JS call returns).
<code>initLayoutResizeCoordinator()</code>	<code>app.ts:116--125</code> — additionally installs a <code>ResizeObserver</code> on four regions (<code>blockly-area</code> , <code>toolbox-column</code> , <code>code-column</code> , <code>viz-dock</code>) so <i>any</i> size change to those elements — including ones with no corresponding JS call at all, e.g. the browser window itself resizing — triggers the same <code>requestLayoutResize(false)</code> path.

6 Command palette

`src/core/ui/commandPalette.ts` (158 lines) is a small, self-contained keyboard-first command surface, deliberately decoupled from knowing what any command actually *does* — it is handed a flat `IdeCommand[]` array by `app.ts` and only ever calls `command.run()`.

Symbol	Role
<code>IdeCommand</code>	<code>commandPalette.ts:1--8</code> — <code>{id, label, category, run, keywords?, shortcut?}</code> .

Symbol	Role
<code>matches(command, query)</code>	<code>commandPalette.ts:21--27</code> — every space-separated query token must appear <i>somewhere</i> in the command’s label $\cup$ category $\cup$ keywords (all lower-cased) — an AND-of-substrings match, not a fuzzy/scored one.
<code>installCommandPalette(commands)</code>	<code>commandPalette.ts:30--158</code> — wires the trigger button, live filtering on input, ArrowUp/ArrowDown selection, Enter to run, Escape to close, click-outside-to-close, and the global shortcut (Ctrl/Cmd+Shift+P or F1 , l.146–154) that opens it from anywhere in the app, restoring focus to whatever was focused before on close (l.105, l.117–118).

The command list itself is assembled in `app.ts` (around `installCommandPalette(commands)` at l.1199) grouping File (new/open/save/export/autosave), Run (CESK/compare/rewrite), and View commands — README’s own description of the palette’s contents.

7 Status bar

`footer.ide-status-bar` (`index.html:381--392`) is a two-group bar: perspective + block count + problems button on the left, autosave status + technical labels (“MiniJava”, “BMJ-Thrasos”) on the right.

Element	Updated by	Trigger
<code>#status-perspective-label</code>	<code>setPerspectiveIdentity</code>	Any perspective change.
<code>#status-block-count</code>	<code>app.ts:221</code> (inside <code>updateCode</code>)	Every workspace mutation — counts via <code>workspace.getAllBlocks(false)</code> .
<code>#status-problems-count</code>	<code>renderProblemsPanel</code> (<code>typeDiagnostics.ts:86--90</code> , documented in <code>bottom-panel-sync.tex</code>)	Every type-check pass.

`#status-perspective` and `#status-problems-button` are themselves clickable: the former opens the perspective picker, the latter calls `showProblems()` (`app.ts:480--483`), which opens the bottom dock directly on the Problems tab — the status bar doubles as a secondary set of shortcuts into the same actions the sidebar/command palette expose.

8 Responsive breakpoints

The shell has three width-based behavioral thresholds, all read live via `window.matchMedia` at the point of use rather than cached once (so a browser window resize takes effect immediately without a separate resize listener re-deriving a “is mobile” flag):

Breakpoint	Where checked	Effect
max-width:1100px	Throughout <code>app.ts</code> (activity buttons, <code>setToolboxHidden</code> , <code>setCodeHidden</code> , drag-resize guards, initial layout at l.1354, l.1358)	The primary "compact" breakpoint: sidebar and inspector become <i>overlay</i> panels (<code>mobile-sidebar-open</code> / <code>mobile-code-open</code> body classes) instead of persistent columns, and are forced closed by default when this width is first detected (l.1354).
max-width:700px	<code>app.ts:1129</code>	A narrower phone-sized threshold for additional layout collapse (per the README's "the shell collapses to tablet and phone layouts").
min-width:901px	<code>app.ts:1364</code>	A recovery threshold on resize <i>back up</i> : past this width, mobile overlay state is cleared.

`compactPanelLayout` (`app.ts:56`) caches the 1100px query's initial value only as a starting default for code that runs before the first layout pass; every subsequent visibility decision re-queries `matchMedia` live, so there is no stale "was compact" state to desync from the real viewport.

9 Theming

Symbol	Role
<code>#theme-toggle</code>	<code>index.html:70--75</code> — a checkbox styled as a sun/moon switch; checked = dark (the app's default, <code>index.html:10</code> 's <code>data-theme="dark"</code>).
<code>createBlocklyTheme(theme)</code>	<code>core/renderer/theme.ts</code> — builds the Blockly <code>Theme</code> object (block-style colors) for 'dark' 'light'; called both for the main workspace (on toggle, <code>app.ts:602</code>) and for every visualization tab's own injected workspace (<code>visualizationPanel.ts:81</code>), so a theme switch is consistent across all Blockly instances in the app, not just the main one.
<code>disposeVizWorkspaces()</code>	<code>visualizationPanel.ts:199--214</code> , called from the theme-toggle handler (<code>app.ts:603</code>) — Blockly workspaces do not support a live in-place theme swap reliably, so the visualization tabs' private workspaces are disposed and (if one was open on a rendered subtree) immediately re-rendered under the new theme, rather than patched.

10 Persisted state

Every piece of layout state that should survive a reload is written to `localStorage` under a namespaced `block-minijava.*` key, read back once at startup. This is the complete inventory, gathered from every `_KEY` constant across the shell:

Key	Governs
<code>block-minijava.autosave.v2</code>	The autosaved workspace blob itself.
<code>block-minijava.theme</code>	Dark/light theme.
<code>block-minijava.autosave.interval</code>	The autosave interval slider’s value.
<code>block-minijava.code.width</code>	Legacy/alternate code-width key (superseded by the layout keys below in current layouts).
<code>block-minijava.layout.code.visible</code>	Inspector column shown/hidden.
<code>block-minijava.layout.sidebar.width</code>	Primary sidebar width.
<code>block-minijava.layout.sidebar.visible</code>	Primary sidebar shown/hidden.
<code>block-minijava.layout.activity</code>	Which activity-bar view is selected.
<code>block-minijava.layout.perspective</code>	The current perspective identity.
<code>block-minijava.layout.bottom.open</code>	Bottom dock open/closed (<code>visualizationPanels</code>).
<code>block-minijava.layout.bottom.height</code>	Bottom dock height.
<code>block-minijava.layout.bottom.tab</code>	Which of the seven bottom-dock tabs was last active.
<code>block-minijava.gc.autoEnabled</code>	CESK tab’s auto-GC checkbox (<code>garbage-collection.tex</code> §4.3).
<code>block-minijava.gc.threshold</code>	CESK tab’s auto-GC allocation threshold.

Note on scope

This note covers the workbench *shell*: regions, activity bar, perspectives, panel sizing, the resize coordinator, command palette, status bar, responsive breakpoints, theming, and persisted layout state. The *content* each region hosts is documented separately: the toolbox in `toolbox-drag-and-drop.tex`, the inspector-column tabs in `typing-code-outline-tabs.tex`, and the bottom-dock tabs in `bottom-panel-sync.tex`. The CESK/A-vs-B machine internals and the garbage collector are covered in `cesk-machine-and-stepper.tex` and `garbage-collection.tex`; the type system and both operational semantics are covered in `typing-and-evaluation-rules.tex`.